

TALLER #3

Desarrollo de un Sistema Web con Django

LOTERÍA BINARIA

Sistema web académico de gestión de sorteos digitales

Estudiante	Cristhian Herrera Nieto
Asignatura	Desarrollo de Aplicaciones Web y Móviles
Fecha de entrega	2 de agosto de 2026
Framework principal	Django 5.2
Bases de datos	SQLite (fase 1) y MySQL 8 (fase 2)

Documento preparado para incorporar capturas reales tomadas mediante pantalla compartida.

Ficha de control de la entrega

Elemento	Información
Nombre del archivo comprimido	HERRERA_NIETO_T4.rar (ajustar al formato solicitado por el docente)
Nombre del documento	Documento_Entrega_Final_Taller4_Loteria_Binaria.docx
Repositorio o rama presentada	https://github.com/Herrera2005/loteriaBinaria-Django.git
Motor activo en la demostración	☐ SQLite
Resultado de pruebas	SQLite: 458 pruebas aprobadas en la última verificación documentada. MySQL: completar con evidencia real.
Estado final	☐ Listo para entrega ☐ Pendiente de capturas ☐ Pendiente de verificación MySQL

Advertencia de integridad académica

Las capturas deben provenir de la ejecución real del proyecto y mostrar pantalla compartida. No se deben fabricar resultados ni afirmar que MySQL está aprobado sin ejecutar la verificación en un servidor MySQL real.

Contenido

Ficha de control de la entrega	1
1. Resumen ejecutivo	3
2. Descripción del sistema	3
3. Justificación, público objetivo y alcance	3
3.1 ¿Por qué se desarrolló?	3
3.2 Público objetivo	3
3.3 Alcance implementado	4
4. Objetivos	4
4.1 Objetivo general	4
4.2 Objetivos específicos	4
5. Investigación y planificación	4
Guía de investigación utilizada	5
6. Uso de inteligencia artificial como apoyo	5
6.1 Propuesta inicial de alcance solicitada a la IA	5
6.2 Apreciación personal	5
7. Arquitectura y módulos	6
Modelo conceptual resumido	6
8. Tecnologías utilizadas	6
9. Desarrollo general del sistema	7
10. Parte 3 — CRUD completo	7
11. Parte 4 — Interfaz con Bootstrap 5.3	11
12. Mejora adicional	15
13. Rasgos principales	18
14. Seguridad, validaciones y buenas prácticas	18
15. Problemas principales y soluciones	19
16. Conclusiones y recomendaciones	19
Anexo A — Comandos principales	21
Preparación en PowerShell	21
Verificación SQLite	21

1. Resumen ejecutivo

Lotería Binaria es un sistema web académico desarrollado con Django para administrar sorteos digitales y las operaciones relacionadas con clientes, vendedores y administradores. La solución integra autenticación, roles, productos de lotería, eventos, boletos, resultados, billeteras, movimientos, solicitudes entre clientes y vendedores, auditoría y programación de series de sorteos.

El proyecto fue construido de forma incremental. En la primera fase se utilizó SQLite para modelar, migrar, probar y validar el sistema. La segunda fase contempla la portabilidad a MySQL 8 mediante la variable de entorno DATABASE_URL, sin eliminar la compatibilidad con SQLite. La interfaz se desarrolló con plantillas de Django y Bootstrap 5.3, complementado con una identidad visual azul oscuro y dorada.

La aplicación supera el mínimo de tres módulos requerido por el taller: el código está organizado en cinco aplicaciones principales —accounts, core, finance, vendors y lottery— y dispone de CRUD, formularios, validaciones, permisos y pruebas automatizadas. La mejora adicional principal es la autenticación con roles y modos de trabajo; además se implementaron operaciones financieras simuladas, trazabilidad, series programadas y resultados automáticos.

2. Descripción del sistema

La aplicación representa un entorno de lotería digital en el que los productos de sorteo pueden manejar combinaciones octales, decimales o hexadecimales. Un administrador configura productos, eventos y series; un vendedor gestiona solicitudes e inventario; y un cliente consulta sorteos disponibles, adquiere boletos y revisa sus resultados y movimientos.

El propósito académico es demostrar comprensión real del ciclo de desarrollo web con Django: configuración del entorno, separación por aplicaciones, diseño de modelos, migraciones, administración de datos, construcción de vistas y rutas, uso de plantillas, formularios, CRUD, autenticación, seguridad, pruebas y cambio de motor de base de datos.

3. Justificación, público objetivo y alcance

3.1 ¿Por qué se desarrolló?

El tema de lotería digital permite reunir en un solo sistema varios retos representativos del desarrollo web: usuarios con distintos permisos, catálogos, eventos con fechas y estados, transacciones, validaciones de combinaciones, historiales, procesos de compra y administración. Por ello resulta apropiado para demostrar el uso de Django más allá de una página informativa.

3.2 Público objetivo

Público	Necesidad cubierta
Clientes mayores de edad	Consultar sorteos, adquirir boletos, revisar combinaciones, resultados y movimientos.
Vendedores autorizados	Atender solicitudes, administrar disponibilidad e inventario y consultar operaciones.
Administradores del sistema	Gestionar usuarios, productos, eventos, series, resultados, auditoría y configuración general.
Docente/evaluador	Verificar el dominio de Django, CRUD, bases de datos, Bootstrap, seguridad,

3.3 Alcance implementado

- Autenticación, registro, cierre de sesión, cambio de contraseña y aceptación de términos.
- Roles CLIENTE, VENDEDOR y ADMINISTRADOR, con control de acceso en el backend.
- CRUD visual de usuarios, perfiles vendedores, productos y eventos.
- Billeteras REAL y VIRTUAL, movimientos y operaciones simuladas.
- Solicitudes cliente–vendedor con estados y trazabilidad.
- Compra de boletos, combinaciones únicas por evento, resultados y acreditación de premios.
- Series de eventos con pausa, reactivación, límites, archivo lógico y resultados automáticos.
- Auditoría, mensajes, páginas de error, diseño responsive y pruebas automatizadas.

4. Objetivos

4.1 Objetivo general

Desarrollar un sistema web funcional con Django que permita gestionar las operaciones principales de una lotería digital, empleando buenas prácticas, tres o más módulos, CRUD completos, una interfaz basada en un framework frontend y portabilidad de SQLite a MySQL.

4.2 Objetivos específicos

1. Configurar un entorno virtual e instalar las dependencias necesarias.
2. Organizar el sistema en aplicaciones Django con responsabilidades claras.
3. Diseñar modelos coherentes con las reglas del negocio y generar sus migraciones.
4. Implementar autenticación, roles, permisos y administración de datos.
5. Construir vistas, URLs, formularios y plantillas reutilizables.
6. Completar operaciones CRUD en al menos tres módulos.
7. Aplicar Bootstrap 5.3 para obtener una interfaz ordenada y responsive.
8. Validar la solución con pruebas automatizadas y verificaciones manuales.
9. Mantener compatibilidad con SQLite y preparar la ejecución sobre MySQL 8.

5. Investigación y planificación

La investigación se realizó por etapas para evitar generar el proyecto completo de una sola vez. Primero se analizó el enunciado del taller y la guía de VideoClub. Luego se contrastó el tema de Lotería Binaria con reglas, flujos, prototipos y documentación de referencia. Finalmente se transformaron las conclusiones en modelos, aplicaciones, formularios, vistas, plantillas, pruebas y scripts de verificación.

Etapas	Pregunta de investigación	Resultado aplicado
Alcance	¿Qué funcionalidades son suficientes para una entrega académica funcional?	Cinco apps, tres CRUD evaluables, login/roles, sorteos y operaciones complementarias.
Modelo de datos	¿Qué entidades representan usuarios, fondos, vendedores y sorteos?	Usuario, términos, wallet, movimiento, perfil, solicitud, producto, evento, boleto, resultado y serie.
Reglas de negocio	¿Qué acciones deben validarse o	Mayoría de edad, unicidad, roles, estados, historial,

	protegerse?	cancelaciones y operaciones idempotentes.
Interfaz	¿Cómo evitar CSS aislado y cumplir con un framework?	Bootstrap 5.3 real con navbar, grid, cards, formularios, tablas, alerts, modal y offcanvas.
Persistencia	¿Cómo conservar SQLite y migrar a MySQL?	DATABASE_URL, configuración portable, utf8mb4, modo estricto y scripts de verificación separados.
Calidad	¿Cómo demostrar que el proyecto funciona?	Pruebas automatizadas, checks, migraciones, seeds idempotentes, smoke test y capturas reales.

Guía de investigación utilizada

- Leer el enunciado y convertir cada criterio de la rúbrica en evidencia verificable.
- Revisar la guía docente para mantener el orden de creación de entorno, proyecto, apps, modelos, migraciones, vistas y templates.
- Definir alcance mínimo y separar lo obligatorio de las mejoras adicionales.
- Diseñar el modelo entidad-relación y comprobar que cubra al menos tres módulos.
- Revisar documentación oficial de Django para modelos, formularios, autenticación, class-based views, migraciones y pruebas.
- Revisar documentación de Bootstrap 5.3 para componentes y comportamiento responsive.
- Investigar diferencias entre SQLite y MySQL que puedan afectar constraints, índices, charset, fechas y concurrencia.
- Implementar por tareas pequeñas, ejecutar pruebas y registrar decisiones antes de continuar.

6. Uso de inteligencia artificial como apoyo

La IA fue utilizada como herramienta de apoyo para proponer alcance, revisar coherencia, redactar prompts, sugerir estructuras, detectar riesgos, preparar pruebas y automatizar tareas repetitivas. Las respuestas fueron contrastadas con el enunciado, la guía docente, los archivos reales y las pruebas del proyecto. El trabajo no se trató como una generación automática completa: cada fase se revisó, adaptó y comprobó en el entorno local.

6.1 Propuesta inicial de alcance solicitada a la IA

Se solicitó una propuesta para un sistema de Lotería Binaria con usuarios, roles, vendedores, sorteos, boletos, resultados y control básico de fondos. La propuesta mínima debía incluir al menos tres aplicaciones, CRUD, plantillas, Bootstrap y base de datos.

6.2 Apreciación personal

Aspecto	Apreciación personal
Correcto o útil	La separación por módulos, la recomendación de un usuario personalizado antes de la primera migración, el uso de formularios Django, la protección de rutas y la planificación por fases.
Exagerado o innecesario	Una arquitectura empresarial, microservicios, pagos reales, APIs externas complejas, aplicaciones móviles completas o infraestructura de producción no eran necesarias para el taller.
Ajustes realizados	Se conservó un monolito Django académico, SQLite en la primera fase, MySQL en la segunda, Bootstrap 5.3, operaciones simuladas y cinco apps con límites claros.
Comprobación personal	El código fue ejecutado, migrado y probado. Los prompts exigieron revisar archivos reales y no inventar modelos, rutas o funciones existentes.

Declaración de autoría

La IA apoyó el análisis, la redacción y la automatización de tareas tediosas. Las decisiones, adaptación del código, ejecución, correcciones, pruebas, capturas y presentación corresponden al estudiante.

7. Arquitectura y módulos

El proyecto utiliza una arquitectura monolítica de Django con plantillas renderizadas en el servidor. config centraliza settings y URLs; las cinco apps encapsulan responsabilidades de dominio. La lógica sensible se desplaza a servicios transaccionales cuando corresponde, mientras que las vistas coordinan solicitudes HTTP, permisos, formularios y respuestas.

Aplicación	Responsabilidad principal	Ejemplos
accounts	Identidad, autenticación y permisos	Usuario personalizado, términos, registro, login, modos y CRUD de usuarios.
core	Navegación y funciones transversales	Inicio, dashboards, auditoría, contexto global y seeds.
finance	Fondos y operaciones simuladas	Wallets, movimientos, recargas, retiros, conversiones, transferencias e inventario.
vendors	Relación cliente-vendedor	Perfiles, solicitudes, asignaciones, confirmaciones, cancelaciones y expiración.
lottery	Núcleo de sorteos	Productos, eventos, series, boletos, resultados, premios y automatización.

Modelo conceptual resumido

Un User puede aceptar TermsVersion, tener Wallets, generar Movements y operar según sus roles. Un VendorProfile representa a un vendedor. Las ConversionRequest y ConversionAssignment coordinan solicitudes entre clientes y vendedores. LotteryProduct define el tipo de producto; DrawEvent representa cada sorteo; Ticket registra una combinación adquirida; DrawResult almacena el resultado; y DrawEventSeries programa eventos futuros.

8. Tecnologías utilizadas

Tecnología	Uso en el proyecto	Justificación
Python 3.12	Lenguaje del backend y scripts.	Amplio ecosistema, legibilidad y compatibilidad con Django 5.2.
Django 5.2.16	Framework web principal.	Incluye ORM, migraciones, forms, auth, admin, templates, pruebas y seguridad.
SQLite	Base de datos de la fase 1.	Configuración simple y adecuada para desarrollo y pruebas académicas.
MySQL 8	Base de datos de la fase 2.	Motor solicitado por el taller; configurado con utf8mb4 y modo estricto.
django-environ	Carga de variables de entorno.	Evita credenciales versionadas y permite seleccionar DATABASE_URL.
mysqlclient	Driver de MySQL.	Conector compatible con el backend MySQL de Django.
Bootstrap 5.3	Framework frontend.	Proporciona grid, componentes, formularios y diseño responsive.
HTML5/CSS/JavaScript	Plantillas e interacción progresiva.	El CSS complementa la identidad; JavaScript no decide permisos ni estados.

Git/GitHub	Control de versiones.	Permite ramas, commits, respaldo y trazabilidad.
PowerShell/Bash	Automatización local.	Scripts de arranque, verificación SQLite y verificación MySQL.

9. Desarrollo general del sistema

El desarrollo siguió una secuencia incremental: preparación del entorno; creación del proyecto y apps; usuario personalizado; modelos y migraciones; administración; autenticación; permisos; CRUD; interfaz; servicios de negocio; pruebas; auditoría; cierre SQLite; y preparación MySQL. Después de cada bloque se ejecutaron checks y pruebas antes de incorporar nuevas funciones.

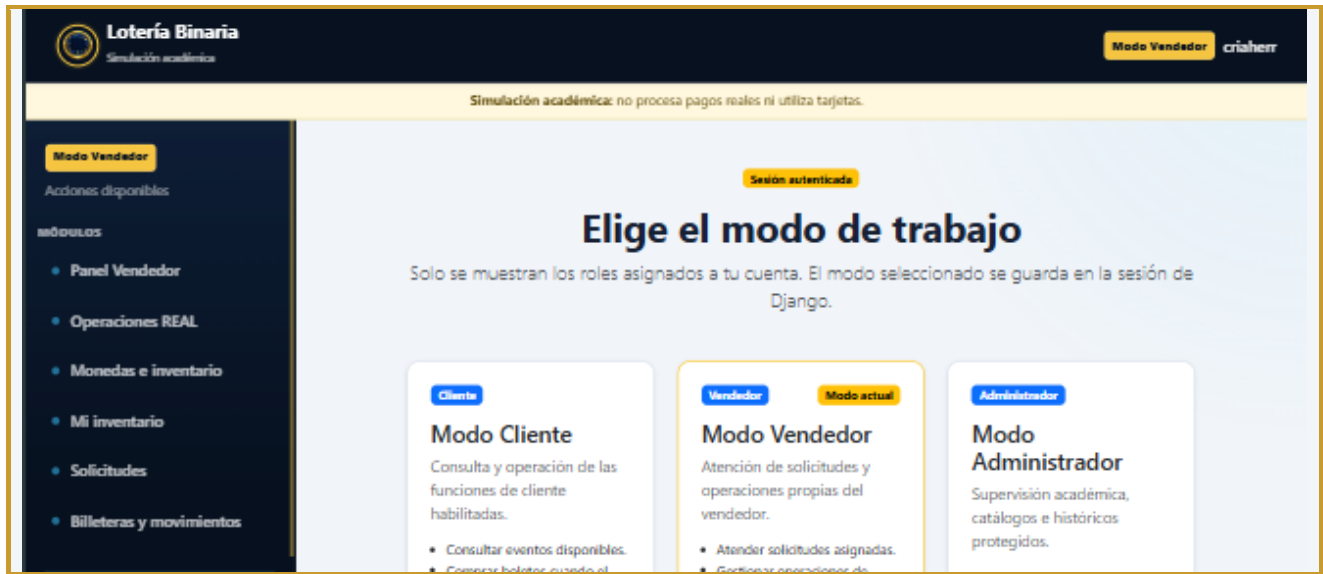
La configuración de base de datos se obtiene de DATABASE_URL. Si no existe, se usa `sqlite:///db.sqlite3`. Cuando el motor es MySQL, se aplican opciones compatibles con `utf8mb4` y modo estricto. Los secretos se colocan en `.env` local y `.env.example` contiene únicamente ejemplos.

10. Parte 3 — CRUD completo

El sistema dispone de operaciones CRUD en más de tres módulos. Para la evaluación conviene demostrar al menos accounts, vendors y lottery, incluyendo listado, detalle, creación, edición y eliminación o desactivación segura.

Módulo	CRUD demostrable	Tratamiento de eliminación
Cuentas	Usuarios: listar, ver, crear, editar y eliminar/desactivar.	Baja física solo sin historia; en caso contrario, desactivación lógica.
Vendedores	Perfiles de vendedor: listar, ver, crear, editar y eliminar/desactivar.	Protección de historial y relaciones.
Lotería	Productos y eventos: listar, ver, crear, editar y eliminar/cancelar según estado.	Las operaciones sensibles conservan trazabilidad.
Complementario	Series: listar, crear, editar, pausar/reactivar y archivar.	Archivo lógico; no destruye eventos generados.

Evidencia 22. CRUD de usuarios — listado



Descripción sugerida: Lista de usuarios con búsqueda, paginación, estados y acciones.

Evidencia 23. CRUD de usuarios — formulario

Lotería Binaria
Simulación académica

Iniciar sesión

Simulación académica: no procesa pagos reales ni utiliza tarjetas.

Registro académico

Crear cuenta

Completa los datos solicitados. La validación y el almacenamiento se realizan en Django.

Nombre de usuario *

Correo electrónico *

Documento *

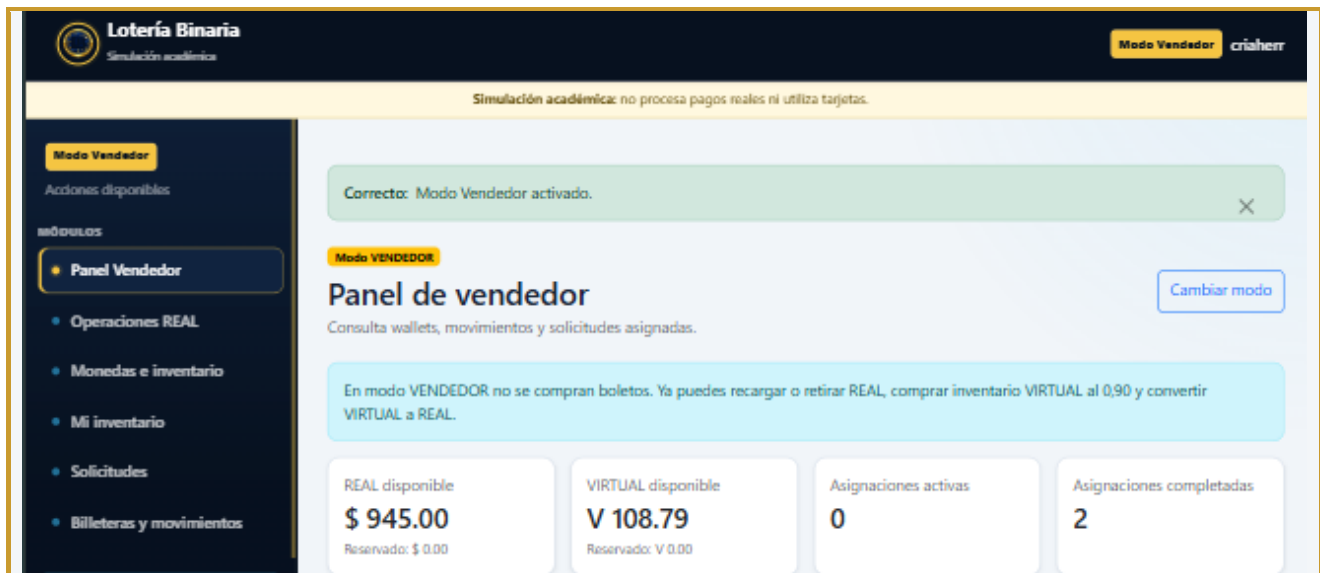
Teléfono

Fecha de nacimiento *

Nombre

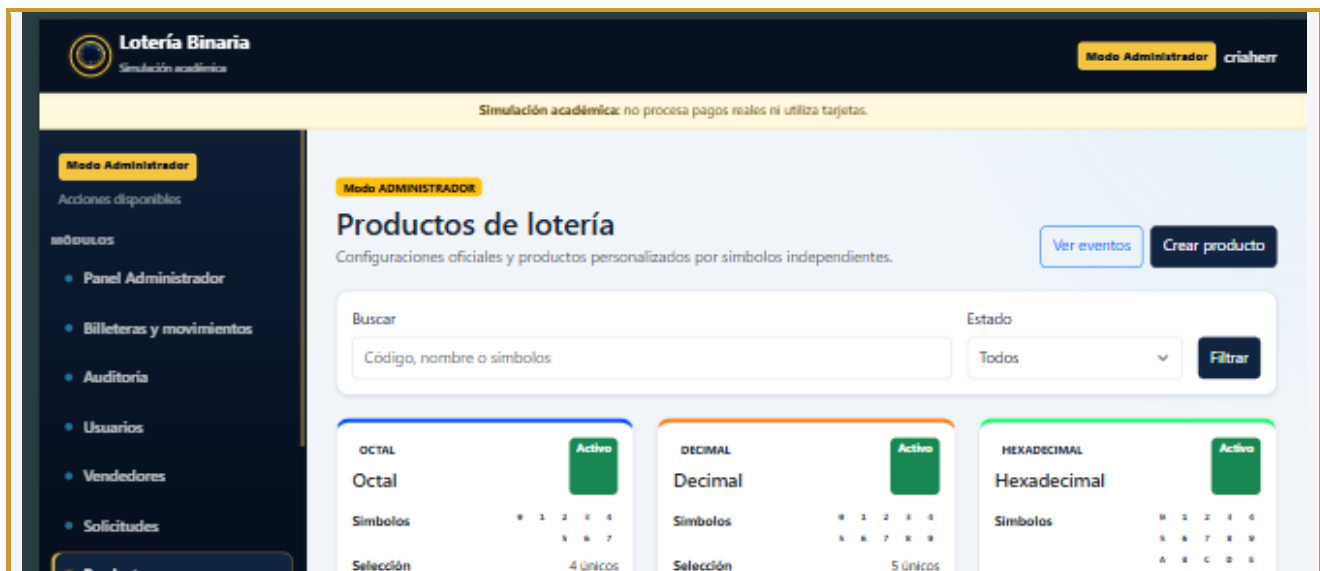
Descripción sugerida: Formulario de creación o edición con validaciones y mensajes.

Evidencia 24. CRUD de vendedores



Descripción sugerida: Listado y formulario o detalle de perfiles vendedores.

Evidencia 25. CRUD de productos de lotería



Descripción sugerida: Listado de productos y formulario Bootstrap.

Evidencia 26. CRUD de eventos de sorteo

Lotería Binaria
Simulación académica

Modo Administrador criaherr

Simulación académica: no procesa pagos reales ni utiliza tarjetas.

Modo ADMINISTRADOR

Eventos de sorteo

Revisa primero los cierres próximos y conserva la trazabilidad de todos los eventos.

Ver productos Crear evento

Buscar Producto Estado Ordenar por

Evento o producto Todos Todos Cierre más próximo Aplicar

Tabla desplazable: en pantallas pequeñas usa desplazamiento horizontal o las flechas cuando la región tenga el foco.

Evento	Producto	Estado	Cierre	Sorteo	Boletos	Acciones
prueba ganador	Octal	Finalizado	02/08/2026 14:09	02/08/2026 14:19	0	Ver Editar
prueba ganador 2	Octal	Finalizado	02/08/2026 14:09	02/08/2026 14:19	0	Ver Editar

Descripción sugerida: Listado/detalle de eventos y formulario de creación o edición.

Evidencia 27. Confirmación de acción sensible

Lotería Binaria
Simulación académica

Modo Administrador criaherr

Simulación académica: no procesa pagos reales ni utiliza tarjetas.

Modo ADMINISTRADOR

Cambiar a Ventas abiertas

Evento: Sorteo Octal #4

Motivo administrativo

Mensaje visible para clientes

Se mostrará en el detalle público del sorteo.

Descripción sugerida: Pantalla de eliminación, desactivación, cancelación o archivo con advertencia y CSRF.

11. Parte 4 — Interfaz con Bootstrap 5.3

La interfaz utiliza Bootstrap 5.3 de manera real. La plantilla base integra navegación, mensajes, offcanvas, modal, grid y componentes reutilizables. Los templates emplean cards, formularios, badges, tablas responsive, paginación, alerts y estados vacíos. El CSS propio complementa la identidad azul oscuro y dorada, sin sustituir al framework.

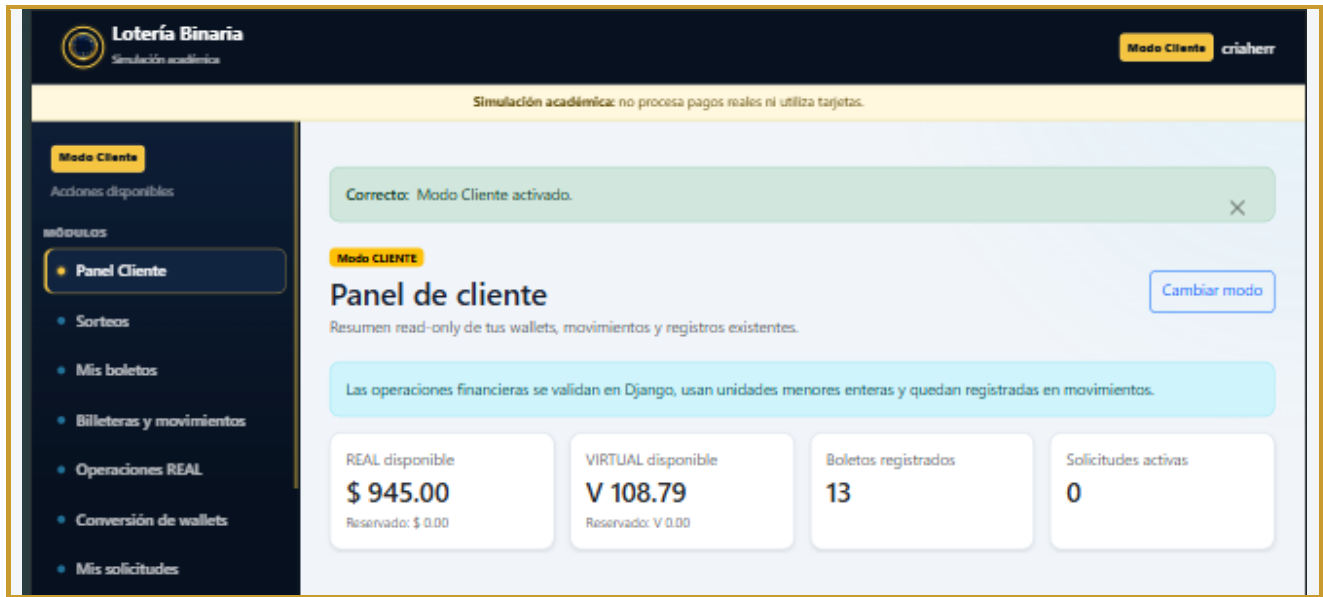
Rasgo visual	Aplicación
Navegación responsive	Navbar/offcanvas adaptado al usuario y al modo activo.
Jerarquía visual	Cards, encabezados, badges y botones con acciones claras.
Formularios	Clases Bootstrap, mensajes de error y ayudas por campo.
Tablas	table-responsive para resoluciones pequeñas.
Accesibilidad	Skip link, foco visible, labels, páginas 403/404 y mensajes comprensibles.
Identidad	Paleta azul oscuro y dorada aplicada de forma consistente.

Evidencia 28. Página de inicio



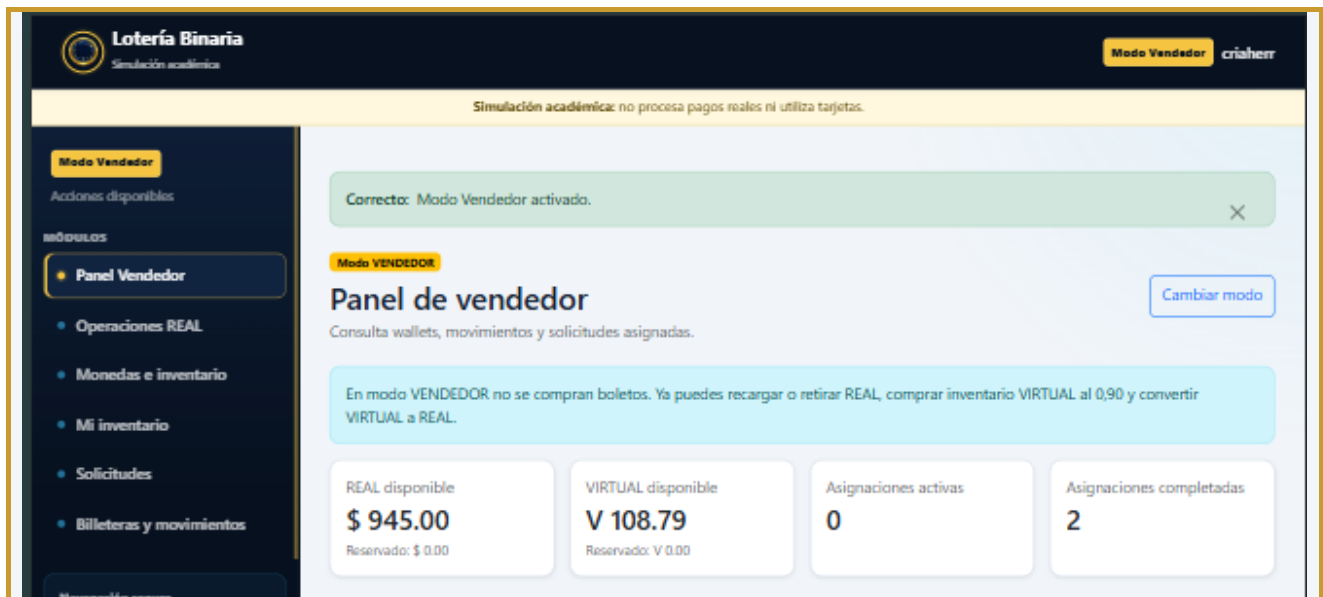
Descripción sugerida: Landing pública mostrando identidad visual, navegación y llamadas a la acción.

Evidencia 29. Dashboard del cliente



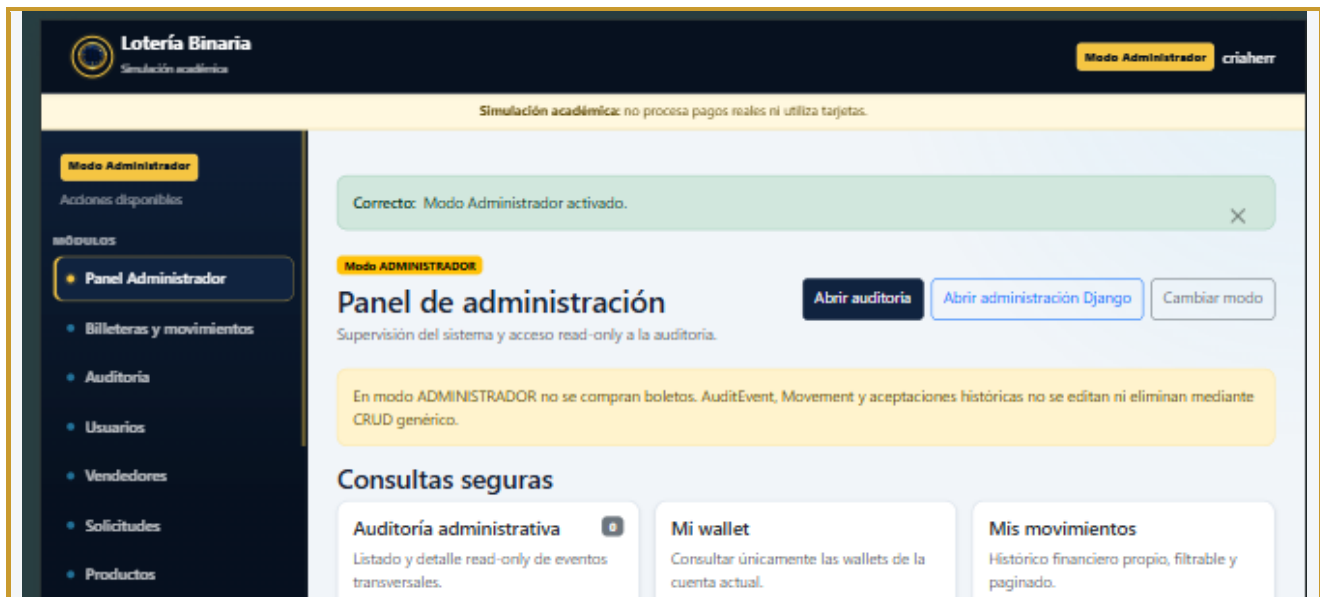
Descripción sugerida: Panel del cliente con accesos, saldos o actividad relevante.

Evidencia 30. Dashboard del vendedor



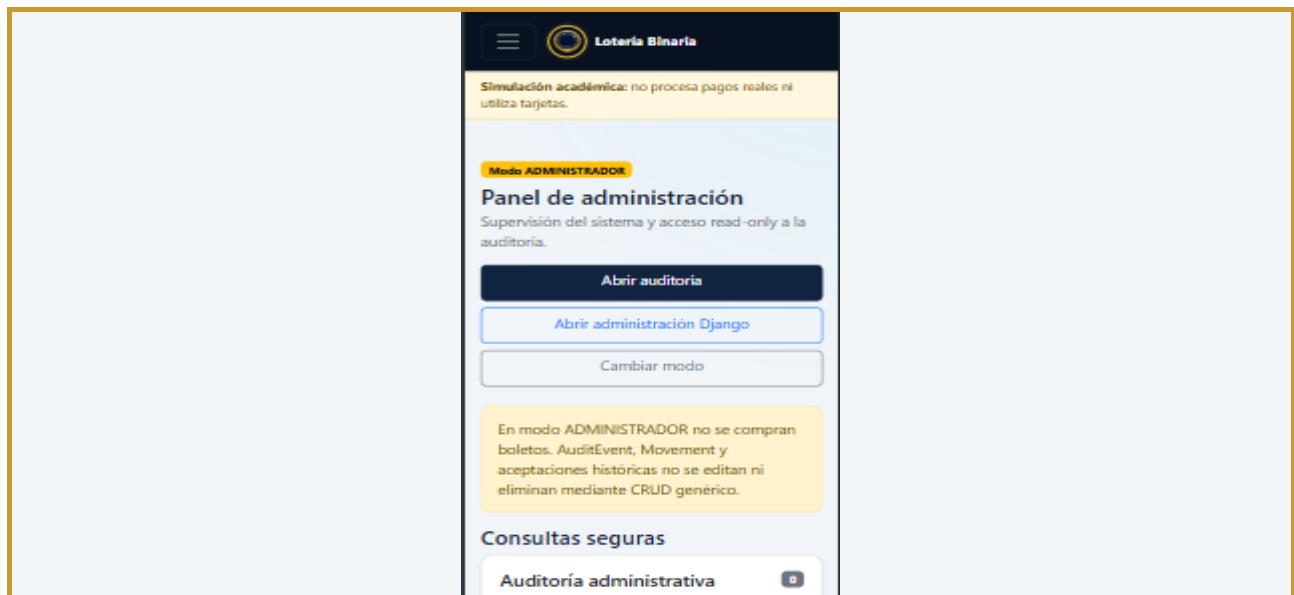
Descripción sugerida: Panel del vendedor con solicitudes, inventario u operaciones.

Evidencia 31. Dashboard del administrador



Descripción sugerida: Panel administrativo con módulos y métricas principales.

Evidencia 32. Diseño responsive



Descripción sugerida: La misma pantalla en vista móvil o herramientas del navegador.

Evidencia 33. Mensajes y validaciones

The screenshot displays the Django Admin interface for editing a user. On the left is a dark sidebar with a menu. The main content area is titled 'Editar usuario' and includes a warning about password security. The form contains fields for 'Nombre de usuario', 'Contraseña', 'Correo electrónico', and 'Documento'. The password field is expanded to show technical details: 'algoritmo: pbkdf2 sha256 iteraciones: 1000000 salto: wCBAYS***** función resumen: A/oZhC*****'. A 'Restablecer contraseña' link is also visible. A red error message 'Este campo es obligatorio.' is shown below the 'Documento' field.

Modo Administrador

Acciones disponibles

módulos

- Panel Administrador
- Billeteras y movimientos
- Auditoría
- Usuarios
- Vendedores
- Solicitudes
- Productos

SESIÓN ACTUAL
criaherr
Modo Administrador

Modo ADMINISTRADOR

Editar usuario

La contraseña se procesa con las herramientas seguras de Django. Esta pantalla administra cuentas y roles de negocio, no permisos técnicos individuales ni superusuarios.

Nombre de usuario *

admin

Requerido. 150 caracteres como máximo. Únicamente letras, dígitos y @/./+/_/.

Contraseña

algoritmo: pbkdf2 sha256 iteraciones: 1000000 salto: wCBAYS***** función resumen: A/oZhC*****
[Restablecer contraseña](#)

Las contraseñas en texto plano no se almacenan, por lo que no se puede ver la contraseña del usuario.

Correo electrónico *

admin@correo.com

Documento *

Este campo es obligatorio.

Descripción sugerida: Alert de éxito/error y errores de formulario visibles.

12. Mejora adicional

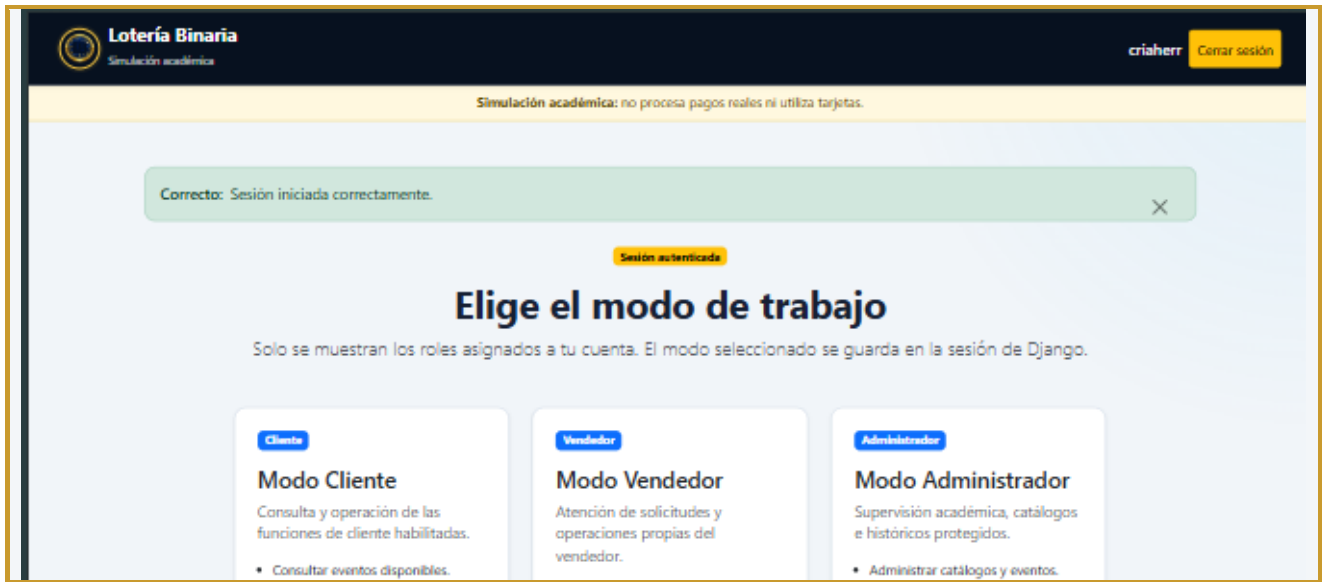
La mejora adicional principal es el sistema de autenticación y autorización por roles. No se limita a ocultar enlaces: las vistas validan en el backend que el usuario esté autenticado, tenga una cuenta activa, posea el rol requerido y haya seleccionado el modo correcto. Esto evita que un cliente acceda a funciones administrativas escribiendo una URL manualmente.

Como ampliaciones complementarias también se incorporaron billeteras REAL/VIRTUAL, movimientos inmutables, solicitudes entre clientes y vendedores, auditoría, series programadas y resultados automáticos. Estas mejoras están justificadas porque aportan seguridad, trazabilidad y automatización a los procesos principales del sistema.

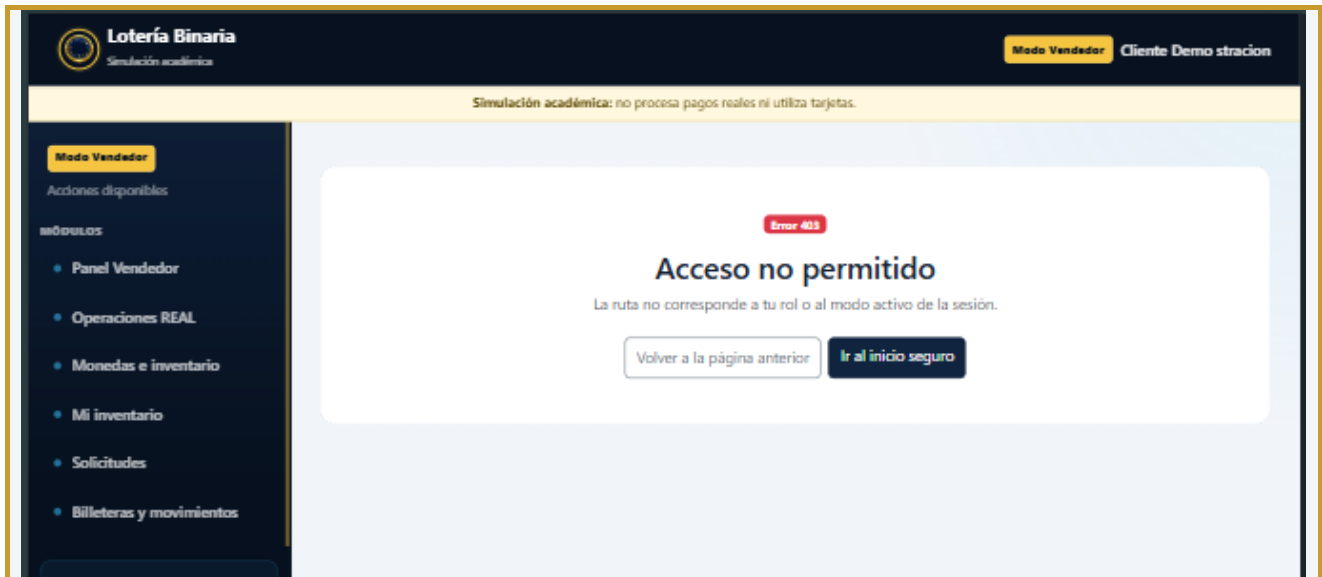
Mejora	Justificación	Resultado
Login y logout	Identifica a cada usuario y protege funciones privadas.	Sesiones Django y redirecciones controladas.
Roles y modos	Separa responsabilidades de cliente, vendedor y administrador.	Permisos verificados en vistas, mixins y políticas.
Auditoría	Permite conocer acciones relevantes.	Eventos consultables por administradores.
Series y automatización	Reduce creación manual repetitiva.	Generación de eventos y resultados automáticos mediante comando.
Pruebas amplias	Reduce regresiones al modificar funciones.	458 pruebas aprobadas en la última verificación SQLite documentada.

Evidencia 34. Inicio de sesión

Descripción sugerida: Formulario de login y mensaje de validación.

Evidencia 35. Selección de modo

Descripción sugerida: Pantalla donde un usuario elige el modo permitido para la sesión.

Evidencia 36. Control de permisos

Descripción sugerida: Página 403 o intento bloqueado al acceder con un rol incorrecto.

Evidencia 37. Series programadas

Crear un evento individual o configurar una serie automática editable. El evento ocurre diez minutos antes de cada sorteo.

¿Cómo desea crear los sorteos?

Evento individual Serie automática

Serie automática

Genera eventos numerados con los mismos parámetros. Puede ser ilimitada o tener un número de generaciones restantes.

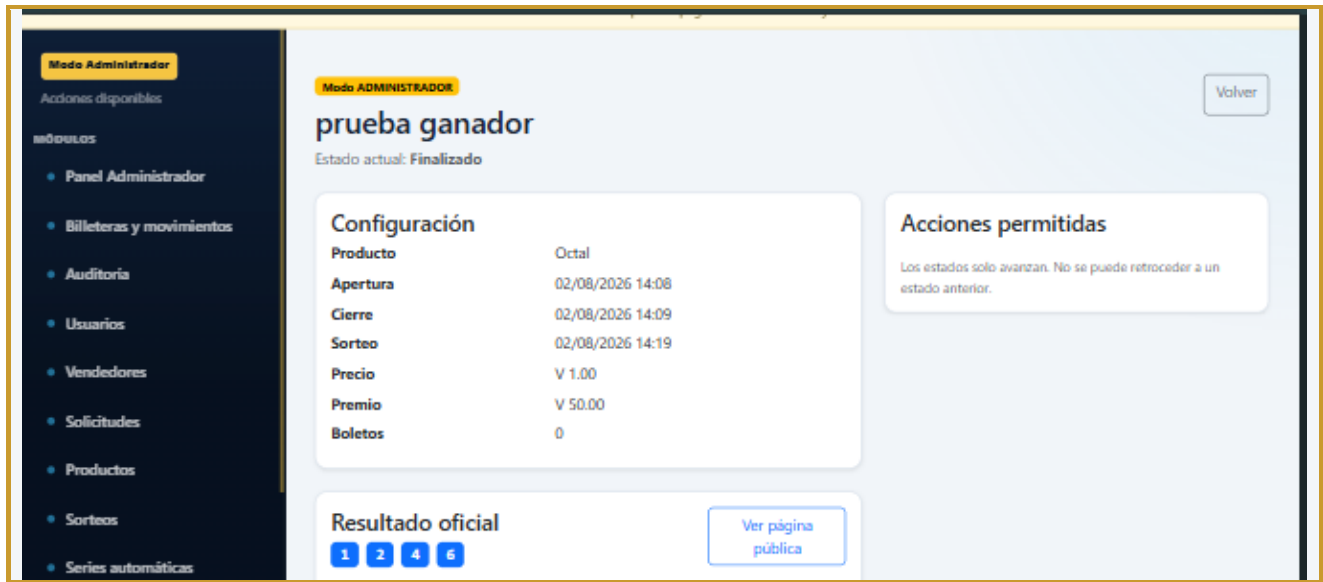
Cambiar una serie afecta únicamente eventos futuros todavía no generados. Los sorteos ya creados conservan sus datos históricos.

Nombre base	Producto

Primer sorteo	Próximo sorteo a generar
dd/mm/aaaa --:--	dd/mm/aaaa --:--
	Al editar, cambia la fecha del siguiente evento aún no generado.
Frecuencia en minutos	Anticipación de ventas en minutos

Descripción sugerida: Lista o detalle de una serie con estado, próxima generación y eventos relacionados.

Evidencia 38. Resultado automático o publicado



Descripción sugerida: Detalle público del resultado y estado de boletos/premios.

13. Rasgos principales

- Usuario personalizado desde antes de la primera migración.
- Mayoría de edad, aceptación de términos y estados de cuenta.
- Roles y modo activo por sesión.
- Cinco aplicaciones con responsabilidades separadas.
- CRUD visuales y formularios con validaciones de servidor.
- Productos octales, decimales y hexadecimales con longitudes definidas.
- Combinación única por evento y control de estados de sorteo.
- Billeteras, movimientos y operaciones simuladas con historial.
- Solicitudes cliente–vendedor con reserva, asignación, confirmación, cancelación y expiración.
- Series limitadas o ilimitadas, pausa, reactivación y archivo lógico.
- Resultados manuales o automáticos y acreditación idempotente.
- Diseño responsive, páginas de error, mensajes y accesibilidad básica.
- Verificación automatizada mediante scripts PowerShell y Bash.

14. Seguridad, validaciones y buenas prácticas

Práctica	Aplicación concreta
Variables de entorno	SECRET_KEY, DEBUG, ALLOWED_HOSTS y DATABASE_URL se separan del código.
Contraseñas	Django almacena hashes; no se guardan contraseñas en texto plano.
CSRF	Los formularios POST incluyen token CSRF.
Autorización	Los permisos se validan en backend, no solo en la interfaz.

Transacciones	Servicios sensibles usan transaction.atomic cuando corresponde.
Historial	Resultados, movimientos y transiciones se protegen contra modificaciones destructivas.
Eliminación segura	Se prefiere desactivar, cancelar o archivar cuando existe historia.
Seeds idempotentes	Los comandos pueden repetirse sin duplicar los datos esperados.
Portabilidad	Las constraints y migraciones se diseñan para SQLite y MySQL.
Separación de responsabilidades	Models, forms, views, services, policies, templates y tests tienen funciones diferenciadas.

15. Problemas principales y soluciones

Problema	Impacto	Solución aplicada / aprendizaje
Definir el usuario personalizado tarde	Cambiar AUTH_USER_MODEL después de migrar puede romper relaciones.	Se creó accounts.User antes de la primera migración y se centralizó su uso.
Permisos por rol y modo	Ocultar botones no evita acceso directo por URL.	Se aplicaron decoradores, mixins, políticas y validación en backend.
Historial frente a eliminación	Borrar usuarios, movimientos o resultados podía destruir trazabilidad.	Se usó PROTECT, inmutabilidad y desactivación/archivo lógico.
Reglas de combinación y estados	Un boleto inválido o una transición incorrecta afecta el sorteo.	Validaciones en modelos/forms/services y pruebas por flujo.
Operaciones financieras concurrentes	Dos operaciones podían leer el mismo saldo.	Servicios atómicos, bloqueos select_for_update y movimientos idempotentes.
Automatización de series	Un error en una serie podía impedir procesar las demás.	Procesamiento aislado por serie y registro de última sincronización.
Compatibilidad SQLite–MySQL	Algunas constraints o comportamientos no son iguales.	Configuración por DATABASE_URL, migraciones nuevas y unicidad portable.
Interfaz responsive	Tablas y acciones podían desbordarse en móvil.	Grid Bootstrap, table-responsive, offcanvas y pruebas por resoluciones.
ZIP con archivos innecesarios	El entorno, .env y base local aumentan tamaño y exponen datos.	Excluir .venv, .env, db.sqlite3, bytecode, logs y staticfiles del paquete final.
Uso responsable de IA	Riesgo de aceptar código inventado o demasiado complejo.	Prompts con restricciones, revisión de archivos reales, pruebas y ajustes personales.

Los retos más importantes no fueron únicamente escribir código, sino mantener coherencia entre modelos, permisos, estados, interfaz, migraciones y pruebas. El principal aprendizaje fue que cada nueva función debe revisarse en todo el flujo: datos, reglas, acceso, presentación y evidencia.

16. Conclusiones y recomendaciones

El desarrollo de Lotería Binaria permitió aplicar de manera integrada los componentes principales de Django: entorno, proyecto, aplicaciones, ORM, migraciones, administración, autenticación, formularios, vistas, rutas, plantillas, archivos estáticos y pruebas. La separación en cinco módulos facilita comprender y mantener el código.

La aplicación cumple ampliamente el alcance funcional mínimo en SQLite y muestra mejoras adicionales reales. La portabilidad a MySQL está contemplada en la configuración y los scripts, pero la entrega debe incluir una ejecución comprobable sobre MySQL antes de declarar esa fase aprobada.

Como recomendación final, se debe realizar una demostración completa con pantalla compartida, capturar cada requisito de la rúbrica, ocultar credenciales, ejecutar las verificaciones finales y generar un archivo comprimido limpio. También conviene conservar el commit final para poder demostrar que las capturas corresponden exactamente a la versión entregada.

Anexo A — Comandos principales

Preparación en PowerShell

```
cd D:\github\LoteriaBinaria-Django

py -3.12 -m venv .venv

.\.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip

python -m pip install -r requirements.txt

Copy-Item .env.example .env

python manage.py migrate

python manage.py seed_baseline

$env:DJANGO_DEMO_PASSWORD = "UnaClaveLocalSegura"

python manage.py seed_demo

python manage.py runserver

deactivate
```

Verificación SQLite

```
python manage.py check

python manage.py showmigrations

.\scripts\verify.ps1
```
